

Pierwszy projekt

W środowisku programistycznym zakładamy **nowy projekt**. Na razie aplikację konsolową. Założenie nowego projektu sprawi, że część danych zostanie wypełniona za nas, co zaoszczędzi trochę czasu.

Przykładowy projekt wygląda następująco:

```
// Dodanie biblioteki. Część rzeczy jest już zrobiona i
ogólnie // dostępna. Nie musimy sami odkrywać koła na nowo. Po
prostu // korzystamy z tych funkcjonalności do własnych celów.
// Biblioteki dodajemy instrukcją #include
// W przykładzie dodajemy bibliotekę na strumień
wejścia-wyjścia. // Potrzebny do m.in. wczytywania danych z
klawiatury oraz // wyświetlania informacji na ekranie (w
konsoli).
#include <iostream>

// Deklarujemy, że będziemy używali przestrzeni nazw „std”. //
Chodzi o wygodę programisty. Bez tej deklaracji musielibyśmy
// za każdym razem pisać „std:” przed np. cout, cin, string,
// endl, itp.
using namespace std;

// Główna funkcja main(). Bez niej nie zadziała żaden program. //
Czasami w nawiasach są jakieś parametry (wyjaśnienie w dalszym)
// rozdziale. Ale z „pustymi nawiasami” też zadziała. // Cały
nasz kod piszemy między klamrą otwierającą, a zamykającą. // (w
przyszłości poznamy także inne techniki na zapis kodu). //
Porównując do schematu blokowego lub pseudojęzyka: // „int
main(){„ to START, „}” na samym końcu to KONIEC algorytmu. int
main()
{
    // W tym miejscu piszemy cały kod programu.
    // Każda instrukcja w kodzie musi być zakończona średnikiem.

    // Na końcu zwracamy wartość 0, aby poinformować,
    // że wszystko powiodło się.
    return 0;
}
```

Przykładowy program

Przykładowy program:

```
#include <iostream>
```

```

using namespace std;

int main()
{
    // To jest komentarz krótki. Po dwóch "/" możemy do końca linii
    // zapisać co tylko chcemy i kompilator nie zwróci na to uwagi.

    /*
    To jest komentarz długi. Nie trzeba "/" w każdej linii.
    */

    // Deklaracja zmiennych. Po lewej stronie typ. Po prawej stronie
    // nazwa. Nazwa powinna być dobrana w sposób inteligentny, aby
    // było wiadomo od razu co dana zmienna robi.
    // Zmienne można deklarować w każdej linii z osobna.
    // Można kilka w jednej linii, jeśli są tego samego typu. //
    // Początkowa wartość zmiennych po takiej deklaracji, jeśli nie
    // przypiszemy wartości początkowych, będzie wynosiła
    // „nie wiadomo”. Deklaracja zmiennej to jedynie zarezerwowanie
    // miejsca w pamięci. Przypadkowe bity się tam znajdujące będą
    // interpretowane jako wartość danej zmiennej.
    int a;
    int b, c;

    // Wyświetlenie komunikatu na ekranie. Komunikaty
    wyświetlamy // z użyciem cudzysłowia. "endl" (end line) to
    wyświetlenie // na ekranie entera (to znaczy przejście do
    nowej linii). // Kolejne wyświetlane elementy oddzielamy
    operatorem "<<". // (w stronę cout'a).
    cout << "Podaj 2 liczby: " << endl;

    // Wczytanie danych z klawiatury. Następnie próba interpretacji
    // kolejnych znaków, czy pasują do konkretnego typu zmiennej.
    // Kolejne wczytywane elementy oddzielamy operatorem ">>". //
    (od strony cin'a w stronę zmiennej).
    cin >> a;
    cin >> b;

    // Jakies operacje na wartościach.
    // Przykładowo dodanie do siebie dwóch liczb.
    // Wynik po lewej stronie znaku "=". Obliczenia po
    prawej. c = a + b;

    // Informacja o wyniku otrzymanym.
    // Kolejne wyświetlane elementy oddzielamy operatorem
    "<<". // (w stronę cout'a).
    cout << "Suma wynosi: " << c << endl;

    return 0;
}

```